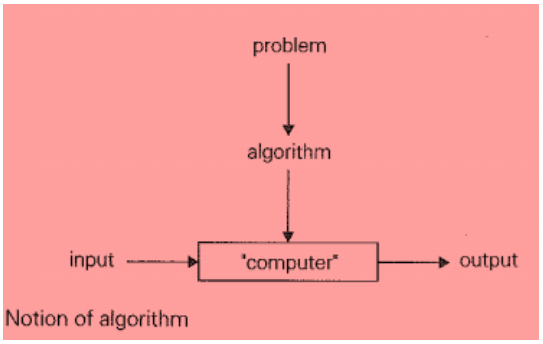
LECTURE 1

What is Algorithm



Notion of algorithm

Finite sequence of set of physical possible steps, performed in correct possible order for solving a computational problem. It takes you from one consistent state to another consistent state whether both of these states may or may not be same. Algorithm must be language independent.

An algorithm is a sequence of unambiguous instructions for solving a problem, i.e., for obtaining a required output for any legitimate input in a finite amount of time.

Properties of Algorithm :-

- It should be finite.
- It should produce at least one output.
- It should take zero or more input.
-It must be deterministic and give the same output for same input.
- Every step must do some work means every step should be effective.

Types of Algorithm :-

- Brute Force Algorithms.
- Recursive Algorithms.
- Backtracking Algorithms.
- Searching Algorithms.
- Sorting Algorithms.
- Divide and conquer Algorithms.
- Hashing Algorithms.
- Dynamic Programming Algorithms.
- Greedy Algorithms.
- Randomized Algorithm

Example :-

Problem1 ---------solution1----------Algorithm1----------Machine1---------Time1

Problem2 ---------solution2----------Algorithm2----------Machine2---------Time2

Scenarios :-

T1 < T2 ---------- why ---------- (Machine M1 is better)

T1 < T2 ---------- why ---------- (Algo1 is better)

T1 > T2 ---------- why ---------- (Algo2 is better)

Note :-

So with the Above Example we came to know , while comparing two different algorithms we make sure problem and machine must be constant.

Algorithms Notations :-

- Algorithm Name (Should be in capital form)
- Parameters (should be enclosed in parenthesis)
- Introductory comment (description and purpose of algorithm)
- Steps(finite steps)
- Comments (Each step starts with a comments, enclose in [ ], defines the purpose of step

-Input statement
- Output statement

Example of Algorithm :-

Algorithm (Step by step method)

Algorithm SUM (No1,No2, Result)

{ This Algorithm is used to read two numbers and print its sum }

Step1 [Read numbers No1 and No2]

Read (No1,No2)

Step2 [Calculate Sum]

Result = No1 + No2

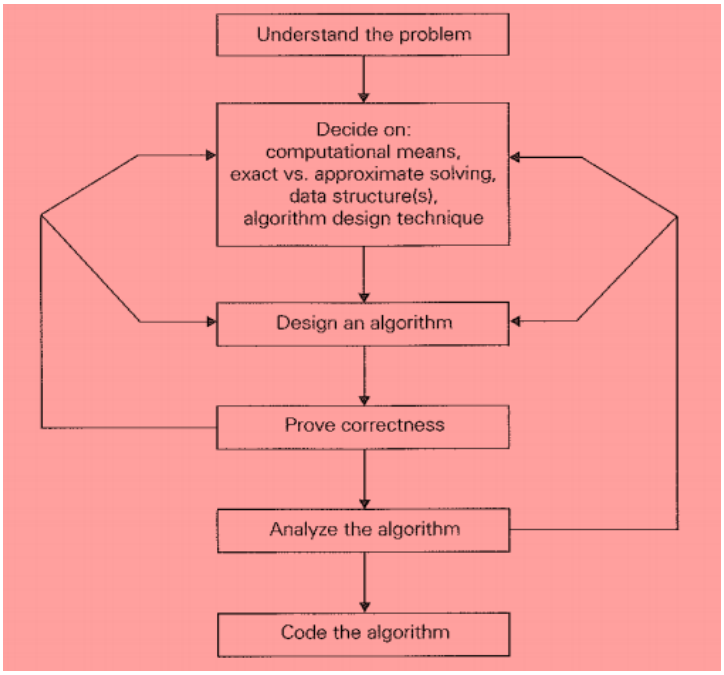
Step3[Display result]

Write (result)

Step4 [Finish]

Exit

Fundamentals of Algorithmic Problem Solving :-



OR

How to design an algorithm :-

Algorithm Design and Analysis Process

Understanding the Problem

From a practical perspective, the first thing you need to do before designing an algorithm is to understand completely the problem given. Read the problem's description carefully and ask questions if you have any doubts about the problem.

Ascertaining the Capabilities of a Computational Device

Once you completely understand a problem. you need to ascertain the capabilities of the computational device the algorithm is intended for.

Algorithm Design Techniques

An algorithm design technique (or "strategy" or "paradigm") is a general approach to solving problems algorithmically that is applicable to a variety of problems from different areas of computing.

Proving an Algorithm's Correctness

Once an algorithm has been specified, you have to prove its correctness. That is, you have to prove that the algorithm yields a required result for every legitimate input in a finite amount of time. For some algorithms, proof of correctness is quite easy; for others, it can be quite complex. A common technique for proving correctness is to use mathematical induction because an algorithm's iterations provide a natural sequence of steps needed for such proofs.

Analyzing an Algorithm

There are two kinds of algorithm efficiency: time efficiency and space efficiency.

1. **Time** efficiency indicates how fast the algorithm runs.

2. **Space** efficiency indicates how much extra memory the algorithm needs.

Coding an Algorithm

So finally we code the above algorithm in your desired language.

1. **Best Case** :-
   While searching the key element of an array, best case is the case when we find the key element at first location of an array.
2. **Worst Case** :-
   While searching the key element of an array, worst case is the case when we find the key element at last location of an array.
3. **Average Case** :-
   While searching the key element of an array, average case is the case when we find the key element in between 2^{nd} and 2^{nd} last location of an array.

4. **Asymptotic Case** :-
   Asymptotic notations was developed to provide the universal way to measure the speed and efficiency of an algorithm. Asymptotic algorithm refers for large input. we measure the behaviour of algorithm when input size reaches infinity.

Notations:-

O(N) → **"Big O"** → Upper bound $O(N^2)$

Ø(N) → **"Theta "** → Tight Bound

$n< 3n <5n$

W(N) → **"Omega"** → Lower Bound

or $\pi(N)$

Note :-

While finding the efficiency of algorithm we are always interested in worst case, which means for analyzing the complexity of algorithm we only care about the highest power and ignore the rest.

STEP FREQUENCY COUNT TABLE :-

Step frequency count method is one of the method to analyze the time complexity of an algorithm. In this method we count the number of times each instructions is executed.
On the basis of such count we calculate the time complexity.

RECURRENCE RELATION :-

A recurrence relation is an equation which represents a sequence based on some rule. Four types are given below.

1. Substitution method

2. Iteration method

3. Recursive tree method

4. Master method

SUBSTITUTION METHOD :-

(i) Forward substitution:-

Forward substitution is a method which uses the initial condition in initial term and generate the next term and keep on repeating the same process until we guess particular formula. We rarely use this method because we will have to guess the formula in this method.

Backward substitution method:-

In this method backward values are substituted recursively. We generally use Backward substitution.

MASTER THEOREM METHOD

Another method to solve recurrence relation, but in this method we can't solve every recurrence relation we only solve specific type of recurrence relations.

LINEAR SEARCH ALGORITHM

Linear Search is defined as a sequential search algorithm that starts at one end and goes through each element of a list until the desired element is found, otherwise the search continues till the end of the data set. Linear Search is also known as s**equential Search. The complexity of algorithm is O(N).**

BINARY SEARCH ALGORITHM

Binary search is a search algorithm used to find the position of a target value within a **sorted** array. It works by repeatedly dividing the search interval in half until the target value is found or the interval is empty. The search interval is halved by comparing the target element with the middle value of the search space. **The complexity of algorithm is O(Log N).**

BUBBLE SORT ALGORITHM

Bubble sort algorithm is the simplest sorting algorithm that works by repeatedly swapping the adjacent elements. This algorithm is not suitable for large data sets as its worst-case time complexity is quite high. **Time complexity of algorithm is $O(n^2)$.**

SELECTION SORT ALGORITHM:-

Selection sort algorithm array is divided in to two sub arrays or lists one is sorted sub array or list and other is unsorted sub array or list, initially we assume the sorted sub array is empty so by taking the smallest element from an unsorted array and bringing it to the **beginning of sorted array** and keep on doing this until the unsorted array is shifted to sorted array. **Time complexity of algorithm is $O(n^2)$.**

INSERTION SORT ALGORITHM:-

Insertion sort algorithm divides array in to two sub lists one is sorted list and other is unsorted list, The idea behind the insertion sort is that first take one element, iterate it through the sorted array and keep on doing till the whole array is sorted.

Insertion sort works similar to the sorting of playing cards in hands. It is assumed that the first card is already sorted in the card game, and then we select an unsorted card. If the selected unsorted card is greater than the first card, it will be placed at the right side; otherwise, it will be placed at the left side. Similarly, all unsorted cards are taken and put in their exact place. **Time complexity of algorithm is $O(n^2)$.**

MERGE SORT ALGORITHM:-

Merge sort algorithm Conceptually, a merge sort works as follows:

1. Divide the unsorted list into n sub lists, each containing one element (a list of one element is considered sorted).
2. Repeatedly merge sub lists to produce new sorted sub lists until there is only one sub list remaining. This will be the sorted list.

Complexity of merge sort algorithm is O(nlogn).

NAIEVE STRING MATCHING

Naive string matching algorithm:-

It is basic string matching algorithm. The naive string matching algorithm compares characters one by one between the pattern and each substring of the text of the same length. The time complexity of the naive string matching algorithm is **O (n-m+1)m**.

In the Naive String Matching algorithm, we check whether every substring of the text of the pattern's size is equal to the pattern or not one by one. Like the Naive Algorithm, the Rabin-Karp algorithm also check every substring. But unlike the Naive algorithm, the **Rabin Karp algorithm** matches the **hash value** of the pattern with the hash value of the current substring of text, and if the hash values match then only it starts matching individual characters. The worst case time complexity of robin Karp algorithm is O(n-m+1)m.

Dijkstra's algorithm :-

Dijkstra's algorithm is used to find the shortest path between the two mentioned vertices of a graph by applying the Greedy Algorithm as the basis of principle. Dijkstra's algorithm can be used to find the shortest route between one city and all other cities. It does not work on graph with negative weights (or edges).The time complexity of Dijkstra's Algorithm is typically $O(V^2)$.

Applications of Dijkstra's Algorithm :-

1. Network routing protocols.
2. Route Planning in transportation network.
3. Communication routing in telecommunication networks.
4. Path finding in video games.

Kruskal Algorithm :-

Kruskal Algorithm also called Minimum spanning Tree. In Minimum spanning tree sum of the weight of all the edges is minimum. Minimum spanning tree of that graph is sub graph such that all the vertices of a graph include in sub graph, it should be connected and have no cycles. (#of vertices -1), suppose we have 9 vertices so MST would be of 8 vertices.

Application of Kruskal Algorithm :-

Computer networks, telecommunications networks, transportation networks, water supply networks, and electrical grids.

Huffman Coding Algorithm :-

Huffman coding Algorithm is a lossless data compression algorithm. In computer science and information theory a Huffman code is a particular type of optimal prefix code that is commonly used for image and video compression. The Complexity of Huffman coding algorithm is O(n logn). where n is the number of unique characters or symbols in the input data. This complexity arises from building the Huffman tree by repeatedly merging nodes and constructing the variable-length codes.

Applications of Huffman coding Algorithm :-

1. Used in image and video compression techniques like JPEG and MPEG.
2. Enabling efficient storage and transmission of multimedia content.
3. Search Engines: Huffman Coding is employed by search engines to optimize the storage and retrieval of data, improving search speeds.
4. Used in conventional compression formats like GZIP, BZIP2, PKZIP, etc.